Testleaf

Always Ahead

</> TECHNICAL DOCUMENTATION

NLP Query Preprocessing Pipeline

A comprehensive code explanation of the modular preprocessing system designed to enhance AI understanding, semantic search, and RAG retrieval accuracy through normalization, abbreviation expansion, and synonym generation.

MODULE 1

abbreviationMapper.js

This file is an **Abbreviation Mapping Utility** used to identify abbreviations and convert them into meaningful full forms. It acts as a preprocessing and NLP normalization layer in AI/RAG systems.

Why It Matters

- Improves AI understanding
- Enhances semantic/vector search
- Helps RAG retrieval accuracy
- Normalizes healthcare/domain-specific terms

Quick Example

UHID → Unique Health ID

OP → Outpatient

IP → Inpatient

Main Functionalities

expandAbbreviations()

Converts abbreviations into full forms

expandAbbreviationsContextual()

Handles abbreviations with multiple meanings

expandHealthcareAbbreviations()

Expands healthcare-related abbreviations

abbreviateText()

Converts full forms back into abbreviations

findAbbreviations()

Finds abbreviations without replacing

smartExpand()

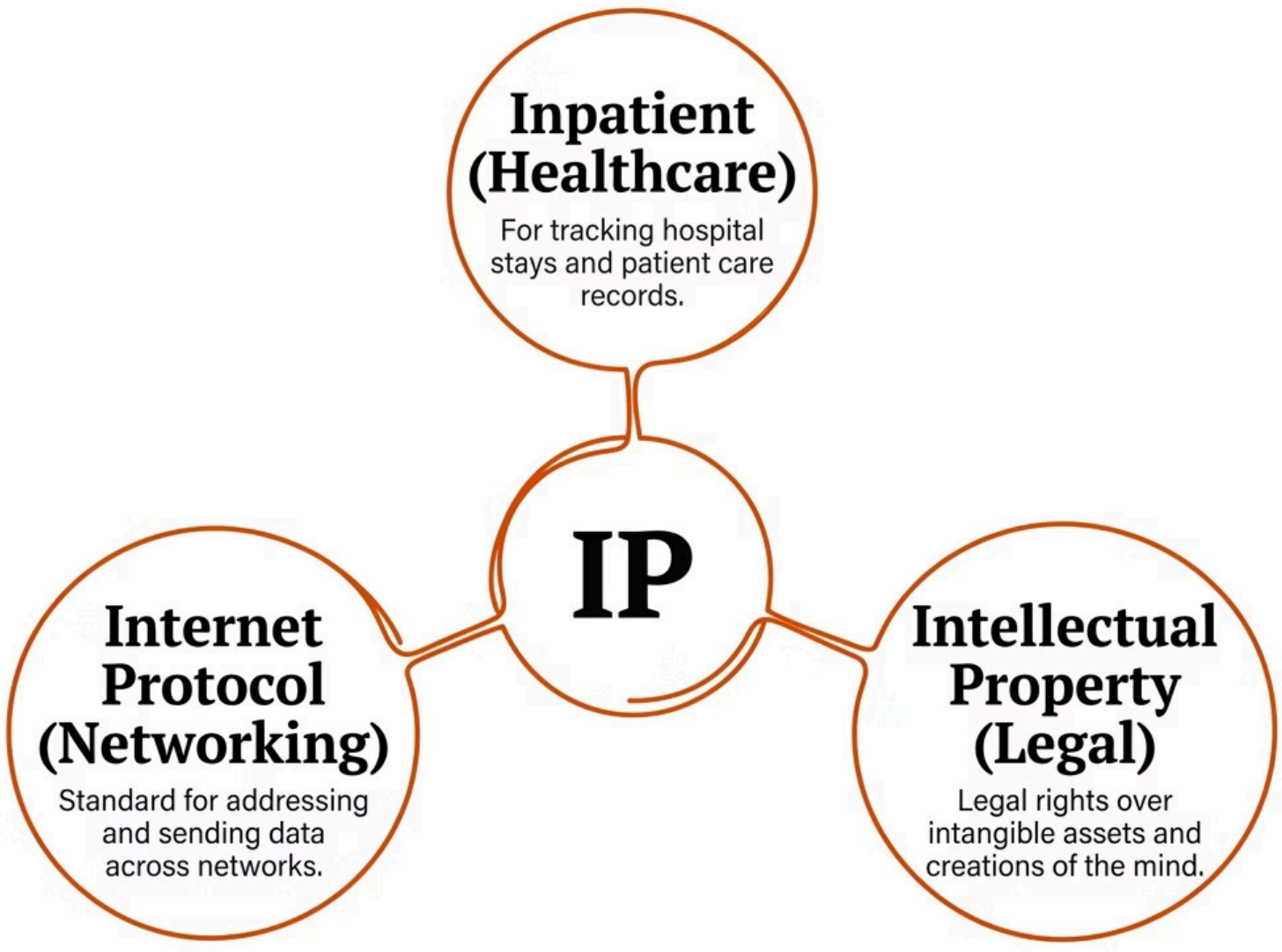
Expands abbreviations only when healthcare context exists

MODULE 1 — DEEP DIVE

Ambiguous Abbreviation Handling

Some abbreviations carry multiple meanings across different domains. Proper contextual disambiguation is critical to avoid incorrect AI interpretation and ensure accurate retrieval.

⚠ Without contextual handling, **"IP address issue"** could incorrectly expand to **"Inpatient address issue"** — a critical misinterpretation in healthcare AI systems.



Avoid Wrong Interpretation

Prevents AI from selecting the incorrect meaning in cross-domain queries

Improve Semantic Understanding

Ensures the correct domain context is applied before expansion

Multi-Domain Search Support

Handles queries that span healthcare, networking, and legal domains

RAG Accuracy

Improves retrieval precision by resolving ambiguity before indexing

MODULE 2

normalizer.js

This file is a **Text Normalization Utility** used to clean and standardize user input before processing. It acts as a foundational preprocessing layer in AI, NLP, Search, and RAG systems.

Main Functionalities

01

normalize()

Removes special characters and normalizes text to a clean format

02

tokenize()

Splits text into individual tokens and words for further processing

03

removePunctuation()

Strips unwanted punctuation marks from raw input

04

normalizeSpacing()

Cleans spaces, tabs, and newlines for consistent formatting

05

extractTestCaseIds()

Standardizes test case IDs to a uniform naming convention

06

normalizeComplete()

Runs the complete preprocessing pipeline end-to-end

Purpose

- Cleans raw user queries
- Improves semantic/vector search
- Standardizes text format
- Enhances AI retrieval accuracy

Transformation Example

Input:

```
TC-027 Patient Registration!!
```

Output:

```
tc-027 patient registration
```

MODULE 3

dictionaries.js

This file is a **centralized dictionary configuration** used for query preprocessing in AI, NLP, Search, and RAG systems. It acts as the centralized NLP knowledge base for the entire preprocessing pipeline.

abbreviationMap

Stores abbreviations and their full forms for expansion during preprocessing.

Example: UHID → Unique Health ID

synonymMap

Stores words with similar meanings to enable semantic query expansion.

Example: doctor → physician

phraseMap

Stores multi-word phrase mappings for business-level query understanding.

Example: password reset → forgot password

preservedStopWords

Stores important stop words that must be retained during normalization.

testCasePrefixes

Stores test case naming patterns for consistent ID standardization.

The **dictionaries.js** file is the single source of truth for all NLP mappings. Updating this file propagates changes across the entire preprocessing pipeline automatically.

MODULE 4

synonymExpander.js

This file is a **Synonym Expansion Utility** used to generate multiple query variations using synonyms. It acts as a semantic query enhancement layer in AI/RAG systems, dramatically improving retrieval coverage.

Purpose

- Improves semantic search coverage
- Enhances AI understanding of intent
- Generates alternate query variations
- Improves RAG retrieval accuracy

Expansion Example

Input Query: doctor appointment

Generated Variations:

- physician appointment
- doctor consultation
- medical officer booking

Main Functionalities

→ expandSynonyms()

Expands words using the synonym dictionary

→ generateSmartCombinations()

Creates intelligent synonym combinations for broader coverage

→ expandPhrases()

Expands multi-word business phrases using phraseMap

→ expandComplete()

Runs the complete semantic expansion pipeline

→ findSynonyms()

Finds all synonyms for a given word

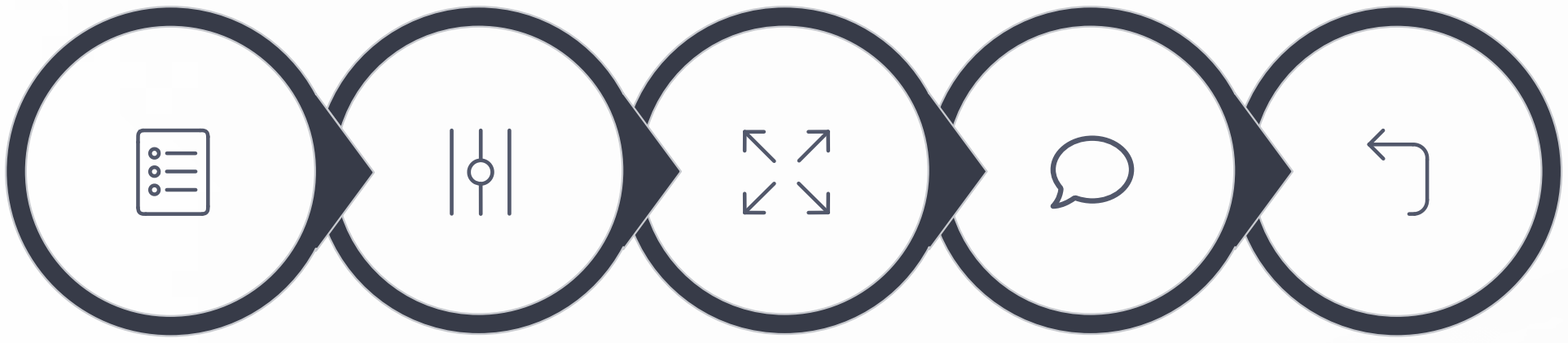
→ expandSynonymsSmart()

Context-aware synonym expansion for precision

CORE ORCHESTRATOR

Query Preprocessor — Main Pipeline

This file is the **central preprocessing pipeline orchestrator** that combines normalization, abbreviation expansion, and synonym expansion into a unified, sequential flow. It is the most critical module in the entire system.



Transformation Example

Stage	Value
Raw Input	TC-027 OP patient registration
After Normalization	tc-027 op patient registration
After Abbreviation Expansion	tc-027 outpatient patient registration
Final Output	TC_027 outpatient patient signup

Main Functionalities

1. **preprocessQuery()** — Main preprocessing pipeline
2. **preprocessQueryQuick()** — Fast preprocessing without synonyms
3. **preprocessQueryComplete()** — Full preprocessing with phrase expansion
4. **analyzeQuery()** — Analyzes preprocessing opportunities
5. **preprocessQueryWithLogs()** — Detailed execution logs for debugging
6. **compareQueries()** — Compares processed query variations

MODULE 6

test-preprocessing.js

This file is a **Command Line Interface (CLI) testing utility** used to test and validate the preprocessing pipeline. It provides developers with detailed visibility into query transformations, execution logs, and debugging capabilities.

Supported CLI Commands

```
node test-preprocessing.js "query"
node test-preprocessing.js --logs "query"
node test-preprocessing.js --analyze "query"
node test-preprocessing.js --examples
```

Main Functionalities

displayResults()

Displays the final preprocessing output in a readable format

displayLogs()

Shows detailed step-by-step execution logs

displayAnalysis()

Analyzes and reports preprocessing opportunities found in the query

runExamples()

Runs a set of predefined sample queries for validation

main()

Handles CLI argument parsing and execution routing

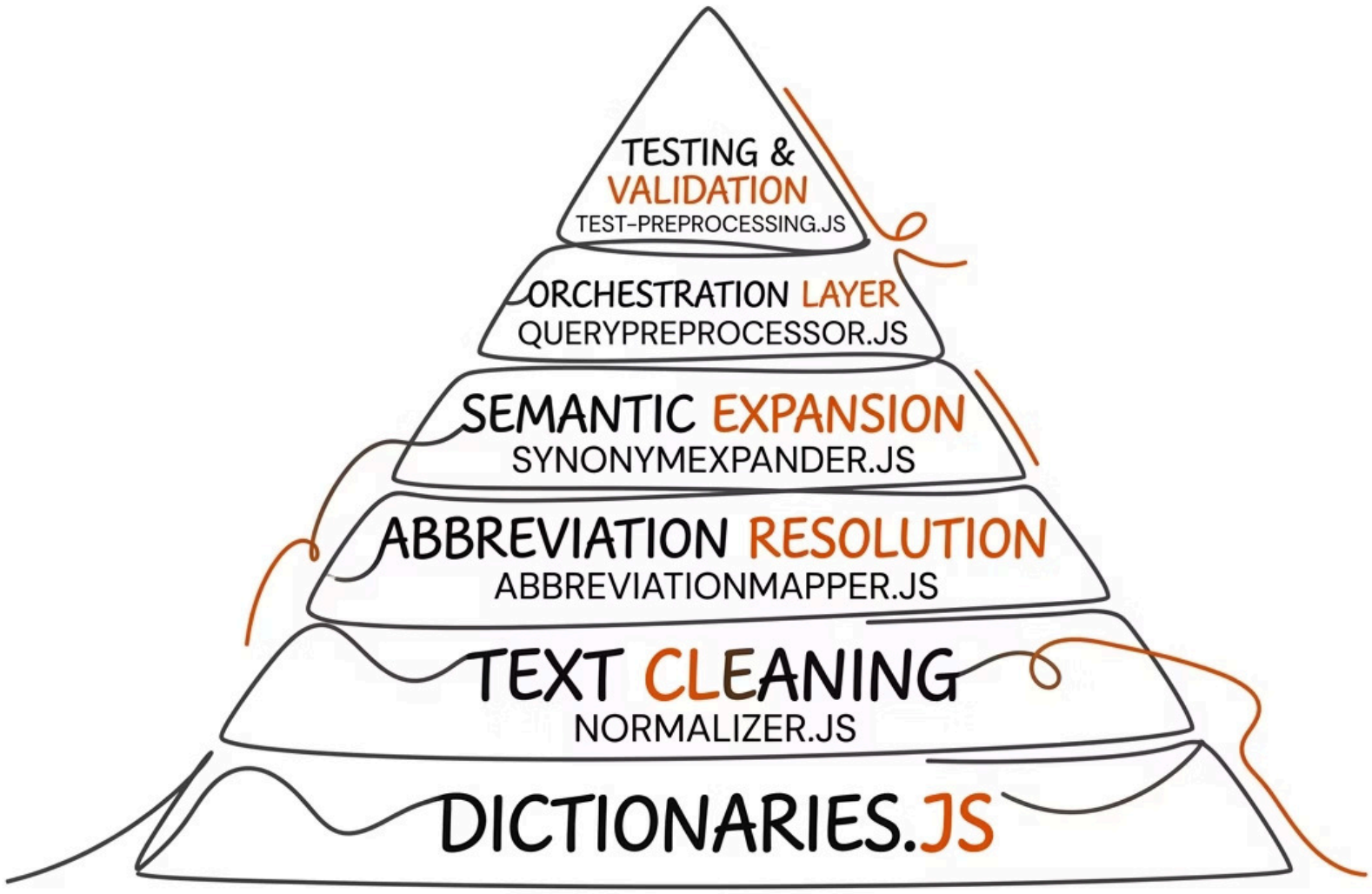
Purpose

- Tests preprocessing flow end-to-end
- Displays preprocessing results clearly
- Shows detailed execution logs
- Analyzes query transformations
- Helps with debugging and validation

Acts as a testing and debugging utility for the entire NLP preprocessing pipeline — essential for development and QA.

System Architecture Overview

All six modules work together as a layered preprocessing system. Each module has a distinct responsibility, and together they form a robust NLP pipeline that powers accurate AI and RAG retrieval.



🗂️ Knowledge Base

dictionaries.js provides the centralized data foundation

🧹 Cleaning

normalizer.js standardizes raw input text

🔤 Expansion

abbreviationMapper.js + synonymExpander.js enrich queries

⚙️ Orchestration

queryPreprocessor.js coordinates the full pipeline

🧪 Validation

test-preprocessing.js ensures correctness and debuggability

Key Takeaways

This preprocessing pipeline is a production-grade NLP system designed to maximize AI and RAG retrieval accuracy through modular, composable utilities. Each component plays a critical role in transforming raw, noisy user queries into clean, semantically rich inputs.

Modular Architecture

Each file has a single, well-defined responsibility — making the system easy to maintain, extend, and test independently.

RAG & Semantic Search Ready

Every module is purpose-built to improve retrieval accuracy in AI/RAG systems by normalizing and enriching queries before they reach the vector store.

Healthcare Domain Optimized

Special handling for healthcare abbreviations (UHID, OP, IP) ensures domain-specific accuracy without sacrificing general-purpose utility.

> The preprocessing pipeline transforms ambiguous, noisy user queries into clean, semantically enriched inputs — acting as the **intelligence layer** between the user and the AI retrieval system.

6

Core Modules

Each with a distinct NLP responsibility

30+

Functions

Covering normalization, expansion, and analysis

4

CLI Commands

For testing and debugging the pipeline

1

Unified Pipeline

Orchestrated by queryPreprocessor.js